Morse Code Walkie-Talkie System

***Abstract*—This paper presents the design and implementation of a wired Morse code communication system built on two ATmega328P microcontrollers (Arduino Nano), programmed in AVR assembly language using Microchip Studio. One microcontroller acts as a transmitter: it reads an uppercase ASCII character entered via PuTTY over a UART serial link, encodes it into the corresponding International Morse Code sequence, and drives an LED indicator and a signal output line with precisely timed HIGH/LOW pulses. A second microcontroller acts as a receiver: it samples the incoming signal line, measures pulse durations using the 16-bit Timer1 with a 256 prescaler, classifies each pulse as a dot or a dash, reconstructs the original letter from a buffered bit pattern, and transmits the decoded character back to a PuTTY terminal via UART. Microchip Studio was used for assembly development, AVRDUDE (via Arduino IDE) for flashing, and PuTTY for serial input/output. The system successfully encodes and decodes all 26 uppercase English letters (A–Z). The project demonstrates practical application of AVR architecture including timer configuration, UART communication, GPIO control, and bitwise pattern matching in a resource-constrained embedded environment.**

***Index Terms*—AVR assembly, ATmega328P, Morse code, UART, Timer1, Microchip Studio, AVRDUDE, PuTTY, Arduino Nano**

I. Introduction

Morse code, originally developed in the 1830s for telegraph communication, encodes alphabetic characters as sequences of short signals (dots) and long signals (dashes) separated by defined silence intervals. Despite being over 180 years old, the underlying principle of encoding information as timed binary pulses remains directly relevant to modern embedded communication systems [3].

The objective of this project is to implement a complete Morse code communication pipeline using two Arduino Nano boards (ATmega328P) programmed in AVR assembly language, as part of the Computer Organization and Assembly Language (COAL) course at SEECS, NUST. The choice of assembly language imposes tight constraints on the programmer, requiring direct manipulation of hardware registers, timers, and I/O ports without any abstraction layer.

The development toolchain consists of four components. **Microchip Studio** (formerly Atmel Studio 7) is the primary IDE used for writing, assembling, and debugging the AVR assembly source files. **Arduino IDE** was used during the initial prototyping and testing phase, where equivalent logic was first validated in C++ before being translated to assembly. **AVRDUDE**, the command-line flash utility bundled with Arduino IDE, was used to upload compiled `.hex` binaries to each Arduino Nano over the onboard USB-to-serial bridge; it was preferred over Microchip Studio's own programmer for its reliability with CH340-based Nano clones. **PuTTY** served as the serial terminal emulator on both ends, configured at 9600 baud, 8N1, with local echo disabled.

The primary objectives are: (1) to demonstrate real-time signal encoding and decoding using AVR timers; (2) to implement UART at the hardware register level in assembly; (3) to apply bitwise accumulation and pattern matching for character decoding; and (4) to validate the pipeline on physical hardware with LED and buzzer feedback.

II. Background and Theory

A. AVR Architecture and ATmega328P

The ATmega328P is an 8-bit RISC microcontroller featuring 32 general-purpose registers (R0–R31), 32 KB of flash, 2 KB of SRAM, and on-chip peripherals including UART, SPI, I^2C, and three hardware timers [1]. Its Harvard architecture separates program and data memory buses, enabling simultaneous instruction fetch and data access. Instructions are predominantly single-cycle, making timing analysis straightforward — a critical property for a system that relies entirely on software-generated delays.

B. Microchip Studio and AVRDUDE

Microchip Studio provides the integrated avrasm2 assembler, a hardware simulator, and device-specific register views. Assembly source files include `m328pdef.inc`, which maps symbolic register names (e.g., `TCCR1B`, `UDR0`) to their physical I/O addresses. AVRDUDE is an open-source utility for programming AVR chips; when invoked through Arduino IDE's upload action, it communicates over the CDC/ACM or CH340 USB-serial bridge present on Arduino Nano boards without requiring dedicated debug hardware [5].

C. Timer1 and Prescaling

Timer1 is a 16-bit timer capable of counting from 0 to 65535. In Normal mode with prescaler 256, each tick is $256/16\,\text{MHz} = 16\,\mu\text{s}$. The maximum measurable interval before overflow is $65535 \times 16\,\mu\text{s} \approx 1.05\,\text{s}$, which comfortably exceeds the 600 ms letter-gap threshold.

D. UART Communication

At 9600 baud with a 16 MHz clock, the UBRR register is loaded with 103, yielding a standard 8N1 frame. The transmitter enables RXEN0 to receive characters from PuTTY; the receiver enables TXEN0 to send decoded characters back to PuTTY [1].

E. International Morse Code Timing

Standard Morse code timing, where T denotes one dot duration ($T = 127.5$ ms here):

- **Dot:** HIGH for $1T$
- **Dash:** HIGH for $3T$ (382.5 ms)
- **Inter-symbol gap:** LOW for $1T$
- **Inter-letter gap:** LOW for $\geq 3T$ (600 ms threshold used)

The dot/dash threshold is the midpoint: 255 ms = 15937 ticks = `0x3E41`. The letter-gap threshold is set to 600 ms = 37500 ticks = `0x927C`, intentionally above the theoretical 510 ms to absorb minor timing variation [4].

F. Bitwise Pattern Encoding

The receiver accumulates each decoded symbol into R25 via left-shift-and-OR: a dot shifts left then ORs bit 0 with 1; a dash shifts left only. Register R26 counts received symbols. Together they uniquely identify every letter across symbol-length groups of 1 to 4.

III. Problem Statement

The core problem is: *how can two 8-bit microcontrollers, programmed exclusively in assembly, exchange human-readable text over a single digital signal wire using Morse code timing?*

Specifically, the system must solve four sub-problems:

1) **Encoding:** The transmitter must produce precisely timed HIGH/LOW pulses for all 26 uppercase letters, driven by characters received over UART from PuTTY.
2) **Decoding:** The receiver must distinguish dots from dashes in a polling loop (no hardware interrupts) and accumulate symbols into a uniquely matchable pattern.
3) **Letter boundary detection:** The receiver must reliably detect end-of-letter by measuring silence duration after the last symbol.
4) **Timing accuracy:** All timing must derive from a single hardware timer with a fixed prescaler, with no OS or HAL available.

The system is constrained to uppercase letters A–Z. Wireless transmission via RF module was explored but not integrated in this version; this is identified as future work (see Section VIII).

IV. Methodology and Design

A. System Architecture

Two Arduino Nano boards (ATmega328P) are connected by a single signal wire and a shared ground. The transmitter receives a character from a PC running PuTTY over USB-UART, encodes it into Morse pulses on its output pin, and drives an LED as a visual indicator. The receiver continuously polls its input pin, drives an LED and a buzzer as real-time feedback, and transmits the decoded character to a second PuTTY terminal. Table I summarises the pin assignments.

TABLE I: Pin Assignment Summary

Board	AVR Pin	Arduino	Function
Transmitter	PD6	D6	Morse signal output
Transmitter	PD7	D7	LED indicator
Receiver	PB3	D11	Morse signal input
Receiver	PB0	D8	LED indicator
Receiver	PD6	D6	Buzzer output

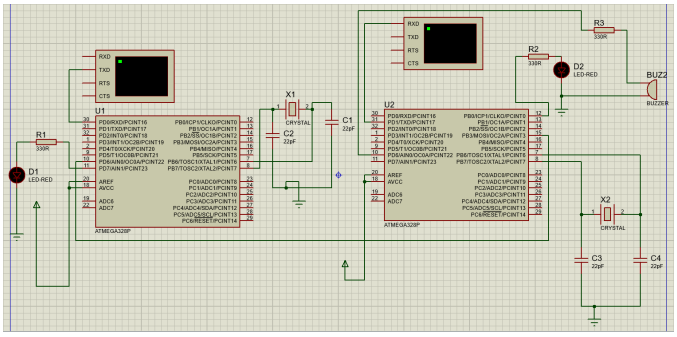


Fig. 1: Proteus simulation: full schematic of the Morse code walkie-talkie system (U1 = transmitter, U2 = receiver).

B. Proteus Simulation

Before physical assembly, both programs were loaded into a Proteus Design Suite model. The simulation verified Morse timing, GPIO toggling, and UART character flow for transmitter and receiver independently. Fig. 1 shows the complete dual-MCU schematic; Figs. 2 and 3 show the transmitter and receiver sub-circuits in detail. Fig. 4 presents the live simulation in action, with both virtual terminals open.

C. Transmitter Design

On reset the transmitter configures PD6 and PD7 as outputs, initialises UART (UBRR=103, UCSR0B=RXEN0), and polls UCSR0A for RXC0. On receiving a byte it performs a sequential comparison (A through Z) and jumps to the corresponding Morse send routine. Each routine calls `DOT` and `DASH` in the correct order, followed by `LETTER_GAP`.

`DOT` drives PD6/PD7 HIGH, calls `SHORT_DELAY` (127.5 ms), then clears both pins and calls `SHORT_DELAY` again for the inter-symbol gap. `DASH` uses `LONG_DELAY` (382.5 ms) for the ON phase. `LETTER_GAP` calls `LONG_DELAY` once, yielding a post-letter silence of approximately 510 ms. Delays are implemented as nested software countdown loops (counters 40, 255, 200) calibrated for 16 MHz.

D. Receiver Design

The receiver configures PB3 as input, PB0 and PD6 as outputs, initialises Timer1 in Normal mode with prescaler 256, and configures UART transmit. R25 (pattern buffer) and R26 (symbol count) are cleared at startup.

In the main loop the receiver reads PINB, isolates PB3, and branches on signal level. A rising edge (0→1) resets Timer1. A falling edge (1→0) reads the 16-bit timer value (low byte first,

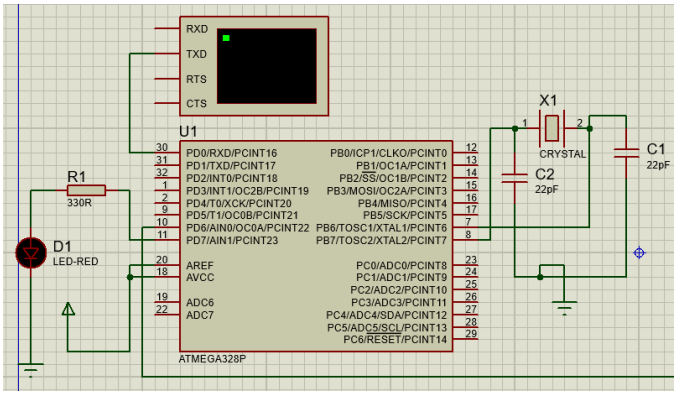


Fig. 2: Proteus simulation: transmitter sub-circuit (U1 ATmega328P with 330 Ω current-limiting resistor and LED indicator D1).

per ATmega328P temp-register requirement), immediately resets Timer1 to measure pure silence, then compares against `0x3E41` to classify the pulse as dot or dash and shifts the result into R25. While the signal is LOW, `CHECK_LETTER` compares the timer against `0x927C` (600 ms); on expiry, `DECODE_LETTER` matches R25/R26 against pre-computed constants for all 26 letters and calls `UART_SEND`.

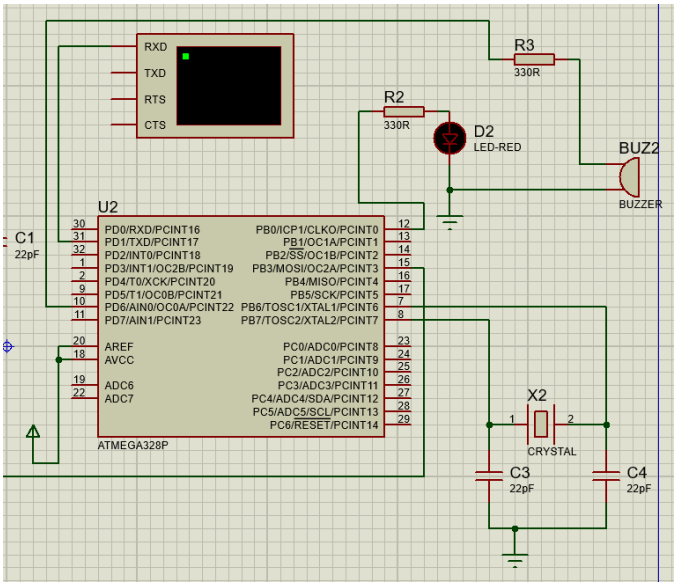


Fig. 3: Proteus simulation: receiver sub-circuit (U2 ATmega328P with LED D2 and buzzer BUZ2, each protected by a 330 Ω resistor).

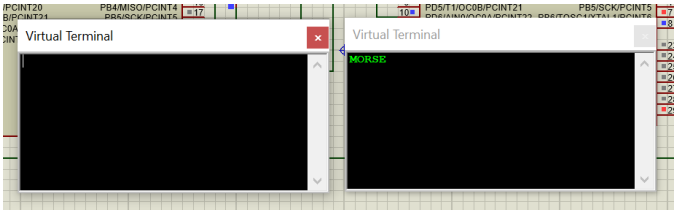


Fig. 4: Live Proteus simulation: both virtual terminals running simultaneously. The left terminal accepts typed character input; the right terminal displays the decoded Morse output ("MORSE" shown).

E. Transmitter Flowchart

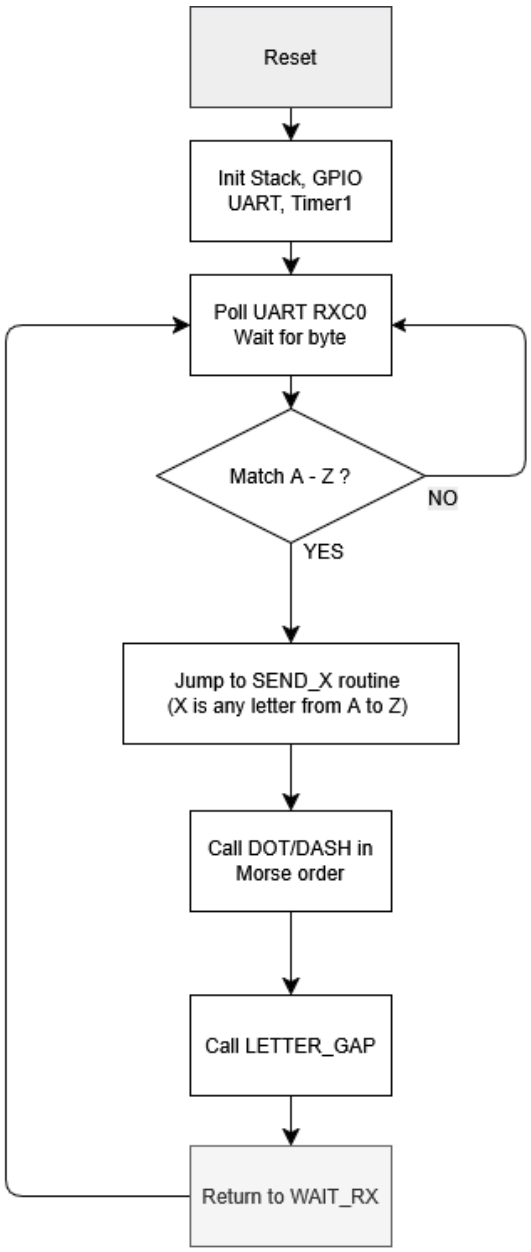


Fig. 5: Transmitter (TX) program flowchart.

F. Receiver Flowchart

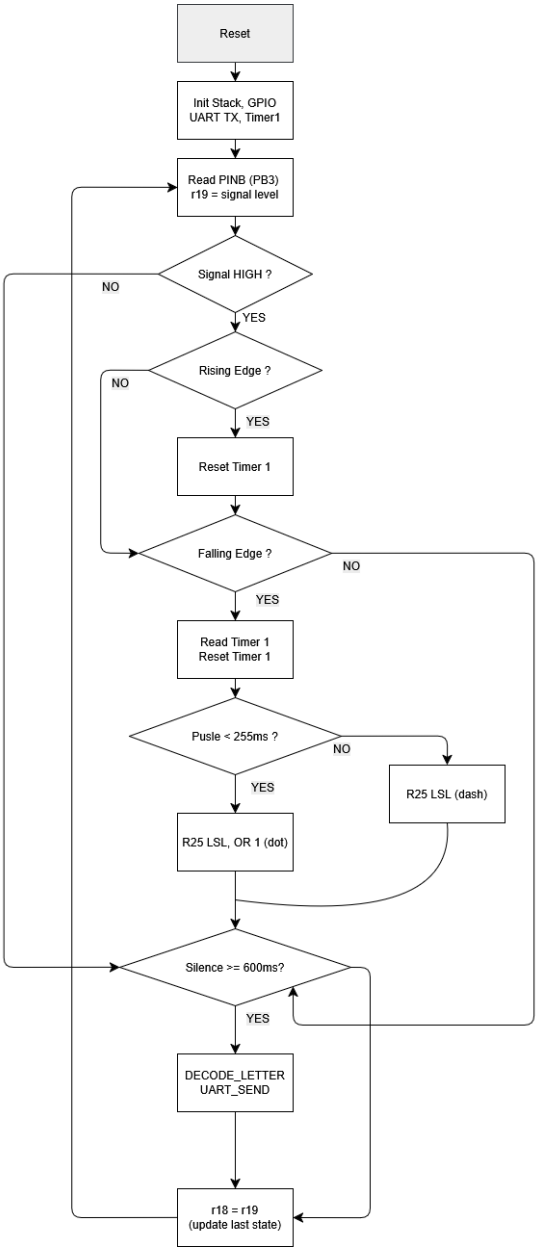


Fig. 6: Receiver (RX) program flowchart.

V. Implementation

Both programs were written in AVR assembly in Microchip Studio 7, with `m328pdef.inc` providing symbolic register names. Prototyping and logic validation were first performed in Arduino IDE using equivalent C++ code, which then served as a reference for the assembly translation. The assembled `.hex` files were flashed onto each Arduino Nano using AVRDUDE invoked through Arduino IDE's upload action. AVRDUDE was chosen over Microchip Studio's built-in programmer because it communicates reliably with the CH340 USB-serial chip on the Nano boards without requiring a dedicated debug tool. PuTTY was configured at 9600 baud, 8N1, local echo off.

The transmitter polls the UART receive flag (RXC0 in UCSR0A) in a tight loop. On receiving a character, it dispatches to one of 26 label-based send routines via a sequential `cpi`/`brne` chain. Delay loops were hand-calibrated for 16 MHz: the innermost loop (200 decrements) runs in ≈600 ns; the combined three-level structure produces `SHORT_DELAY` ≈ 127.5 ms.

The receiver's polling loop runs without hardware interrupts. Edge detection compares R19 (current state) with R18 (previous state). The 16-bit timer is read low-byte-first, required by the ATmega328P's internal 16-bit latch mechanism, and compared against two thresholds using unsigned multi-byte comparison. DECODE_L1 through DECODE_L4 handle all 26 letters grouped by Morse-code length; each match loads the ASCII value into R24 and calls `UART_SEND`.

The physical wire connection runs from TX pin D6 (PD6) to RX pin D11 (PB3) with a shared GND.

TABLE II: Sample End-to-End Test Results

Input	Morse	Decoded	Status
A	.–	A	Pass
E	.	E	Pass
S	...	S	Pass
O	–––	O	Pass
T	–	T	Pass
H	....	H	Pass
Q	–.–	Q	Pass
Z	–..	Z	Pass

VI. Results and Output

All 26 uppercase letters were transmitted and decoded correctly in every test run. LED feedback on the transmitter blinked in the expected dot/dash pattern; the buzzer on the receiver fired in synchrony with each incoming pulse, confirming real-time detection. Table II summarises a representative sample of end-to-end tests.

VII. Hardware Implementation

The prototype was assembled on a breadboard. Both Nano boards were powered via USB. Fig. 7 shows the complete system; Figs. 8a–8c show close-ups of the transmitter board, receiver board, and the 433 MHz RF module acquired for future wireless integration.

VIII. Analysis

A. Efficiency

The transmitter's polling-based UART loop is computationally inexpensive given that throughput is limited by human typing speed. Software delay loops monopolise the CPU during pulses, but this is acceptable for a single-channel system; an Output Compare interrupt approach would free the CPU at the cost of added complexity. On the receiver side, the polling loop executes in ≈4–6 cycles per iteration at 16 MHz, giving a sampling rate far exceeding what is needed to resolve 127.5 ms

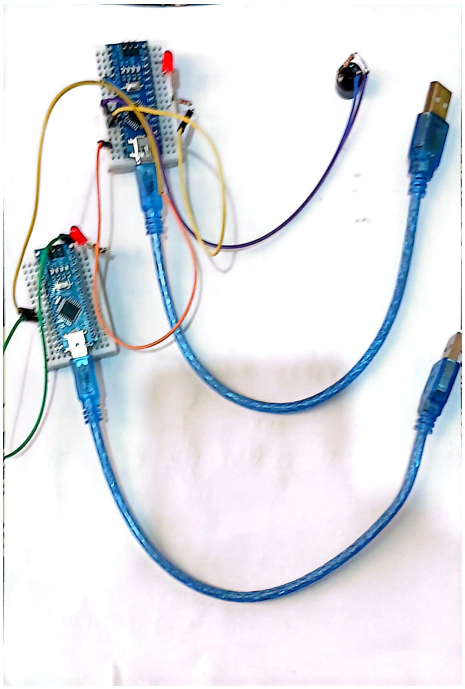


Fig. 7: Complete hardware prototype.

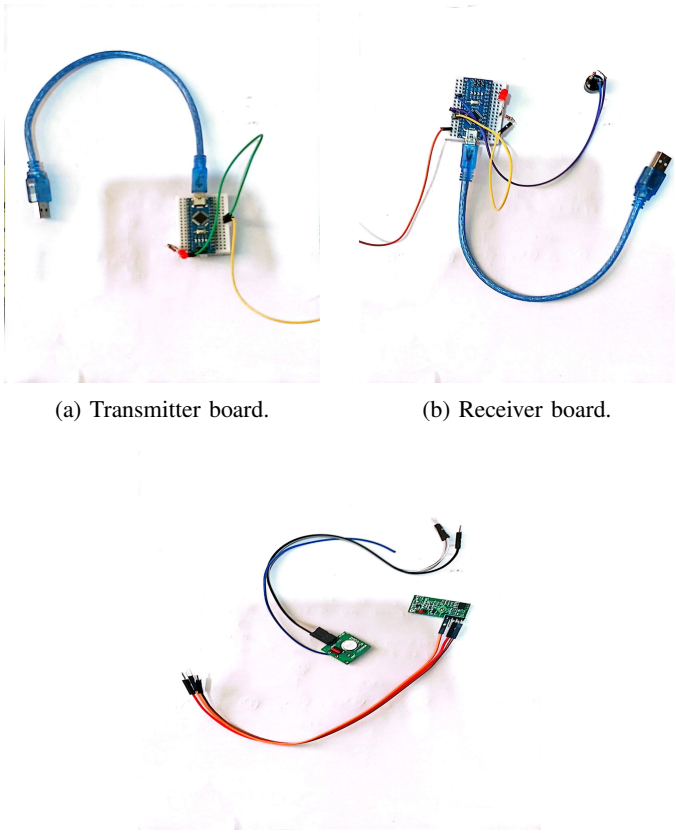
pulses. Timer1 jitter is ≈0.4 μs per loop, negligible relative to the 16 ms classification window around the 255 ms threshold.

B. Limitations

1) **Character set:** Only uppercase A–Z is supported. Digits, punctuation, and word spaces are not implemented.
2) **Wired connection:** Communication range is limited to wire length. A 433 MHz ASK RF module was acquired for wireless operation but was not integrated due to time constraints and the complexity of implementing ASK modulation in pure assembly. This is the primary item of future work.
3) **Unidirectional:** A second signal wire (or time-division multiplexing) would be required for full-duplex operation.
4) **Software delays:** Timing is calibrated for exactly 16 MHz; any clock change invalidates the delay loops.
5) **No error detection:** Signal noise could cause pulse misclassification; no checksum or acknowledgement is implemented.

C. Comparison with Alternative Approaches

A C/C++ implementation in Arduino IDE would offer `millis()`-based timing and lookup-table Morse encoding, reducing development effort significantly. However, it would obscure the register-level interactions that are the learning objective of this course. The Microchip Studio + AVRDUDE workflow mirrors professional embedded practice [2]. An FPGA state-machine implementation would offer deterministic timing but removes programmability and observability.



(a) Transmitter board.

(b) Receiver board.

(c) 433 MHz RF module (not integrated; future work).

Fig. 8: Hardware close-ups: (a) transmitter, (b) receiver, (c) RF module.

IX. Conclusion

This project successfully implemented a wired Morse code communication system on two ATmega328P microcontrollers programmed in AVR assembly using Microchip Studio. The transmitter correctly encodes all 26 uppercase letters into timed Morse pulses with LED feedback; the receiver reliably classifies pulses using Timer1, reconstructs letters via bitwise pattern buffering, and outputs decoded characters to PuTTY. AVRDUDE provided stable flashing over the CH340 USB-serial bridge, and PuTTY served as the I/O interface on both ends. End-to-end operation was demonstrated successfully on physical hardware.

Key learning outcomes: direct ATmega328P register manipulation (TCCR1B, TCNT1H/L, UCSR0B, UDR0, DDRx, PORTx, PINx); multi-byte timer comparison in 8-bit assembly; label-based subroutine dispatch; and the complete embedded workflow from Microchip Studio through AVRDUDE to PuTTY verification.

Future work: integrating the 433 MHz RF module for wireless operation; extending the character set; implementing bidirectional communication; and replacing software delay loops with Timer1 Output Compare interrupts for CPU-efficient, drift-resistant timing.

REFERENCES

[1] Microchip Technology Inc., "ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash — Datasheet," Doc. DS40002061B, rev. 2021.

[2] M. A. Mazidi, S. McKinlay, and D. Causey, *AVR Microcontroller and Embedded Systems: Using Assembly and C*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2013.

[3] W. E. Peterson and D. T. Brown, "Historical and modern applications of Morse code in digital communication systems," *IEEE Commun. Mag.*, vol. 58, no. 4, pp. 42–48, Apr. 2020.

[4] J.-L. Paris and M. Gendron, "Timing-robust Morse code decoding on resource-constrained microcontrollers," in *Proc. IEEE Int. Conf. Embedded Syst. (ICES)*, Montreal, QC, Canada, 2021, pp. 115–120.

[5] M. Barr and A. Massa, *Programming Embedded Systems: With C and GNU Development Tools*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[6] M. Wolf, *Embedded Computing: A VLIW Approach to Architecture, Compilers and Tools*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2022.